Alexandria University
Faculty of Engineering
Computer and Systems Engineering Department

Natural Language Processing

Lab #1 Report

Part 1: N-Gram Language Model
Part 2: Text Classification (Emotion Recognition)

Team Members

Name	ID
Omar Khaled Hussien	22010962
Yassin Asaad Ahmed	22011349
Abdulrahman Khaled Nour-Eldin	22010877

Contents

Abstract **2**

1 Part 1: N-Gram Language Model **2**

1.1 Background . . . 2
1.2 Dataset . . . 2
1.3 Data preprocessing . . . 2
1.4 Unigram baseline . . . 3
1.5 Extending to bigrams, trigrams and general n-grams . . . 3
1.6 Laplace smoothing . . . 4
1.7 Text generation . . . 4
1.8 Evaluation: perplexity . . . 5
1.8.1 Analysis . . . 6
1.8.2 Supplementary experiments . . . 7
1.9 Part 1 conclusions . . . 8

2 Part 2: Text Classification **8**

2.1 Dataset . . . 8
2.2 Preprocessing for classification . . . 9
2.3 Part 2.1: Naïve Bayes classification . . . 9
2.3.1 Model . . . 9
2.3.2 Implementation . . . 10
2.3.3 Comparison with scikit-learn . . . 10
2.4 Part 2.2: Logistic regression . . . 10
2.4.1 Feature representation: binary bi-gram presence . . . 10
2.4.2 Softmax (multinomial) logistic regression from scratch . . . 11
2.4.3 Comparison with scikit-learn . . . 12
2.5 Part 2.3: Confusion matrix and evaluation metrics . . . 13
2.5.1 Definitions . . . 13
2.5.2 Implementation . . . 13
2.5.3 Validation against scikit-learn . . . 14
2.6 Results on the test set . . . 14
2.7 Discussion of the classification results . . . 17

Abstract

This report documents our work on Lab 1. In **Part 1** we build an N-gram language model from scratch (unigram up to $n = 10$) with Laplace smoothing and a text-generation function, train it on two Shakespeare tragedies (*Julius Caesar* and *Macbeth*) and measure perplexity on a third (*Hamlet*). In **Part 2** we classify English sentences into six emotions (sadness, joy, love, anger, fear, surprise). We implement Naïve Bayes, multinomial logistic regression on binary bi-gram features, and the confusion matrix with precision, recall and F1 (per class and macro-averaged), all with NumPy only, and compare each against its scikit-learn counterpart.

The main findings are: (i) on this small corpus, with add-one smoothing, the unigram model has the lowest perplexity (186.5 on Hamlet) and perplexity *grows* with n until it plateaus near 1 873 for $n \geq 5$, because of data sparsity and because Laplace smoothing moves too much probability mass to unseen events; (ii) our Naïve Bayes is *identical* to scikit-learn's (100% prediction agreement) and obtains the best accuracy (0.7319) but a low macro-F1 (0.5348) because it almost never predicts the rare classes; (iii) scikit-learn's `SGDClassifier` obtains the best macro-F1 (0.5533), while our from-scratch logistic regression (accuracy 0.6244, macro-F1 0.4528) is still under-trained after 15 epochs; and (iv) our confusion-matrix and precision/recall/F1 code matches scikit-learn exactly.

1 Part 1: N-Gram Language Model

1.1 Background

A language model assigns a probability to a sequence of words $w_1, \ldots, w_m$. By the chain rule,

$$P(w_1, \ldots, w_m) = \prod_{i=1}^{m} P(w_i \mid w_1, \ldots, w_{i-1}). \tag{1}$$

Conditioning on the full history is impossible to estimate from finite data, so an n-gram model makes the *Markov assumption*: a word depends only on the previous $n - 1$ words,

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-n+1}, \ldots, w_{i-1}). \tag{2}$$

With $n = 1$ we get the unigram model, $n = 2$ the bigram model, $n = 3$ the trigram model, and so on.

1.2 Dataset

We use the Gutenberg corpus available in NLTK. As required, the **training corpus** is `shakespeare-caesar` plus `shakespeare-macbeth`, and the **evaluation corpus** is `shakespeare-hamlet`. Using a different play for evaluation measures how well the model generalizes to unseen text by the same author, rather than how well it memorizes its training data.

1.3 Data preprocessing

Table 1 lists the preprocessing steps and the reason for each.

Table 1: Preprocessing pipeline for the language model.

Step	Reason
Sentence splitting (`sent_tokenize`)	An n-gram model predicts a word from the previous $n-1$ words. Splitting into sentences lets us mark where sentences start and end with `<s>` and `</s>`, so that the model learns how sentences begin and end and context does not leak across sentence boundaries.
Lowercasing	*The* and *the* are the same word; otherwise counts are split across variants and the data becomes sparser.
Tokenization and punctuation removal	One regular expression, `[a-z0-9]+(?:'[a-z]+)?`, extracts word tokens (letters and digits, with an optional inner apostrophe so *o'er* stays one word). Punctuation, brackets and other special characters are dropped.
Stop-word removal (optional)	Implemented behind a flag but **disabled**. Stop-words (*the, of, to*, ...) form a large part of real text and carry the grammatical structure that a language model is supposed to learn; removing them would make the text unnatural and the perplexity not comparable with a normal language model.

Three special tokens are used: `<s>` (start-of-sentence padding; $n-1$ copies are prepended for an n-gram model), `</s>` (end of sentence) and `<UNK>` (any word outside the training vocabulary).

Table 2: Corpus statistics after preprocessing.

	Sentences	Tokens	Unique words
Training (Caesar + Macbeth)	3 057	38 365	5 218
Evaluation (Hamlet)	2 355	29 699	4 813

A key observation is that **4 520 of the 29 699 Hamlet tokens (15.2%) never occur in the training data**. This motivates both the `<UNK>` token and smoothing.

1.4 Unigram baseline

The unigram model assumes that words are independent:

$$P(w_1, \ldots, w_m) = \prod_{i=1}^{m} P(w_i), \qquad P(w) = \frac{C(w)}{N}, \tag{3}$$

where $C(w)$ is the training count of w and N is the total number of training tokens including the end-of-sentence markers. In our data $N = 41\,422$ (38 365 words plus 3 057 markers) and the raw vocabulary has 5 219 types. The most probable tokens are `</s>` (0.0738), *the* (0.0296), *and* (0.0283), *i* (0.0208), *to* (0.0200), *of* (0.0167), *you* (0.0143) and *that* (0.0124). The unigram model ignores word order completely, so it is the weakest model and a natural baseline. Without smoothing, an unseen word such as *hamlet* gets probability 0.

1.5 Extending to bigrams, trigrams and general n-grams

Under the maximum-likelihood estimate (MLE),

$$P(w_i \mid w_{i-n+1}^{i-1}) = \frac{C(w_{i-n+1}^{i})}{C(w_{i-n+1}^{i-1})}. \tag{4}$$

Our class `NGramLM(n, k, min_count)` works for any n and stores two count tables: the counts of full n-grams $C(\text{context}, w)$ and the counts of their $(n-1)$-word contexts $C(\text{context})$. A third table maps each context to the counter of words that followed it; it is used by the generation function. Every sentence is padded with $n-1$ copies of `<s>` and one `</s>`.

Vocabulary and unknown words. The vocabulary contains the training words that occur at least `min_count`$= 2$ times, plus `</s>` and `<UNK>`, giving $V = 2\,357$. Words seen only once in training are *replaced by* `<UNK>`, and at test time every word outside the vocabulary is also mapped to `<UNK>`. This design choice matters: if `<UNK>` never appeared in training, it would receive almost zero probability and the 15.2% of unknown Hamlet tokens would be punished far too hard. Training on `<UNK>` lets the model learn a realistic probability for "some word I do not know".

1.6 Laplace smoothing

The problem. Under MLE every n-gram not seen in training gets probability 0. A single such n-gram makes the probability of the whole test sentence 0 and the perplexity infinite, and the problem worsens rapidly as n grows because long n-grams almost never repeat.

Laplace (add-one) smoothing. We pretend every possible n-gram occurred one extra time:

$$P_{\text{Laplace}}(w_i \mid \text{ctx}) = \frac{C(\text{ctx}, w_i) + 1}{C(\text{ctx}) + V}. \tag{5}$$

The denominator grows by V because we added 1 to each of the V possible next words, so the probabilities still sum to 1. In the code this is the parameter k (general add-k smoothing is $\frac{C+k}{C(\text{ctx})+kV}$); $k = 1$ is Laplace and $k = 0$ is plain MLE.

Effect on a test sentence. For the first long Hamlet sentence (the title line, *"the tragedie of hamlet by william shakespeare 1599 actus primus"*) the trigram MLE model gives $\log P = -\infty$, while the Laplace-smoothed trigram gives $\log P = -71.73$: smoothing turns an impossible sentence into a merely unlikely one.

1.7 Text generation

To generate text we sample one word at a time:

1. Start from the context `<s>` … `<s>` ($n-1$ start tokens) or from a user-supplied prompt.
2. Take the last $n-1$ words as the context and obtain the distribution $P(w \mid \text{ctx})$.
3. Sample the next word from it with NumPy's random generator.
4. Append the word, slide the context window, and repeat until `</s>` is produced or the length limit is reached.

The function supports two modes. With `smoothed=False` (default) it samples only among the words *actually observed* after the context, and *backs off* to a shorter context if the context was never seen; this yields the most readable text. With `smoothed=True` it samples from the full Laplace-smoothed distribution over the whole vocabulary, which is the "pure" model.

Table 3: Text generated by the models trained on Caesar + Macbeth (seeded; outputs copied from the notebook).

Model	Generated text
Unigram	*cymber so not of fellowes so night hot bru*
Unigram	*compell'd i crowne finde being tent rise i play and execution was men of thriue for me i shall in see but <UNK> is soundly*
Bigram	*call timely on your sword there's nothing can entreat a walking crowne to our loue brutus should i thanke you marke that her giue thee*
Trigram	*call em let me finde him fortune and more terrible and caesar shall not liue looke with a new father wife*
4-gram	*call em let me see is not the leafe turn'd downe where i left <UNK>*
Trigram, prompt "the king"	*the king hence to day bru*
Trigram, full Laplace distribution	*liue wel choose wee fleance humble strong holds mothers amisse shewes mile found sorrow fight kingdomes cauldron highnesse decree english shape faint lightning wonders whither*

Reading the output. The unigram text is a bag of random frequent words with no structure. Bigram text has locally plausible word pairs. Trigrams and 4-grams start copying whole phrases from the plays: the training data is small, so higher-order models memorize it, and short outputs such as *"goes the king hence to day"* appear because the model reached `</s>` early. Tokens such as *bru* are speaker abbreviations from the script. The fully smoothed trigram generates mostly random vocabulary words, because most trigram contexts are unseen and Laplace smoothing then spreads probability almost uniformly over the vocabulary.

1.8 Evaluation: perplexity

Definition. Perplexity measures how surprised the model is by unseen text (lower is better). For a test set with N predicted tokens,

$$\mathrm{PP} = \exp\Big(-\frac{1}{N} \sum_{i=1}^{N} \log P(w_i \mid \mathrm{context}_i) \Big) = P(w_1, \ldots, w_N)^{-1/N}. \tag{6}$$

Intuitively, a perplexity of 200 means the model is as uncertain as if it had to choose uniformly among 200 words at every step. In the implementation we sum log-probabilities (a product of thousands of small probabilities would underflow), pad each sentence with `<s>`/`</s>`, count `</s>` as a predicted token but not `<s>`, and map out-of-vocabulary test words to `<UNK>`. Models are trained on Caesar + Macbeth and evaluated on Hamlet.

Results. Table 4 and Figure 1 show the perplexity for $n = 1, \ldots, 10$.

Table 4: Perplexity on Hamlet for $n = 1, \ldots, 10$ (training on Caesar + Macbeth, $V = 2\,357$).

n	**Laplace ($k = 1$)**	**No smoothing (MLE)**
1	186.5	182.9
2	483.4	∞
3	1 446.8	∞
4	1 815.7	∞
5	1 867.2	∞
6	1 871.7	∞
7	1 872.3	∞
8	1 872.8	∞
9	1 873.2	∞
10	1 873.2	∞

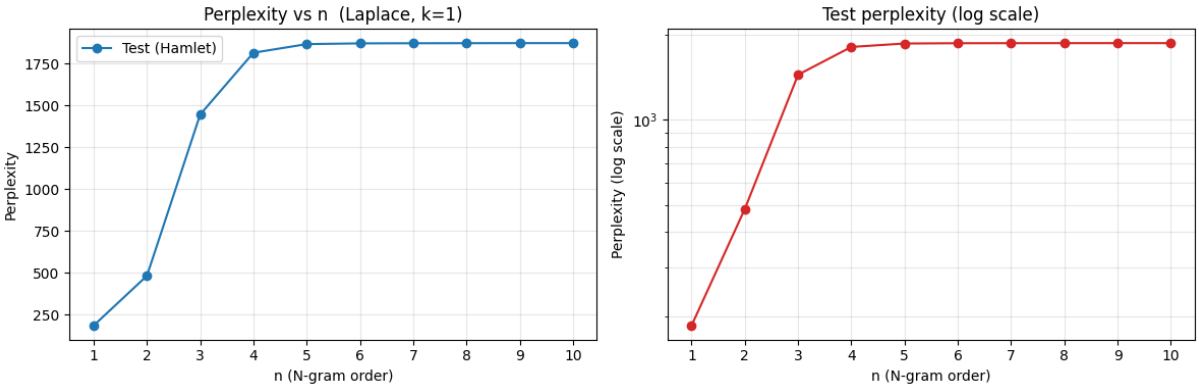


Figure 1: Test perplexity versus n (Laplace, $k = 1$): linear scale (left) and logarithmic scale (right). The best value is $n = 1$ (186.5).

1.8.1 Analysis

More context made the model worse, not better: the unigram baseline wins and perplexity rises from 186.5 ($n = 1$) through 483.4, 1 446.8 and 1 815.7 to a plateau of about 1 873 for $n \geq 5$. This is not a bug; it has three causes.

1. **Data sparsity.** The training set is tiny (about 38 000 tokens, $V = 2\,357$). Table 5 shows that 76.5% of the distinct bigrams seen in training occur only once and 92.4% of the trigrams; and 31.3% of the bigram positions in Hamlet, 70.7% of the trigram positions and 88.2% of the 5-gram positions were never seen in training.

2. **Laplace smoothing gives too much mass to unseen events.** For a context seen only a few times the denominator is $C(\text{ctx}) + V$ with $V = 2\,357$, so the words that really followed that context keep only a tiny share of the probability, while almost all the mass is spread over thousands of words that never followed it. For an unseen context the model is simply uniform, $P \approx 1/V$.

3. **The plateau for $n \geq 5$.** At this order practically every Hamlet context is unseen (except the `<s>`… `<s>` start contexts), so the model behaves like a uniform distribution over the vocabulary and perplexity stops changing. It stays below $V = 2\,357$ only because the start-of-sentence contexts are still seen.

Table 5: Sparsity of the n-gram statistics (computed with the notebook's code). "Unseen" is the percentage of n-gram positions in Hamlet whose n-gram never occurs in the training data.

n	Distinct n-grams in training	Occurring once (%)	Unseen in Hamlet (%)
1	2 357	0.0	0.0
2	21 583	76.5	31.3
3	34 387	92.4	70.7
4	36 957	96.6	85.6
5	37 315	97.3	88.2

The role of smoothing. Without smoothing, perplexity is infinite for every $n \geq 2$; only the unigram model stays finite (182.9) because `<UNK>` absorbs the unknown words. A single unseen n-gram makes the whole test probability zero. Smoothing is therefore what gives the model a way to generalize to unseen sequences, at the price of some accuracy on seen ones. The unsmoothed unigram is slightly better than the Laplace unigram (182.9 vs. 186.5) because there are no unseen unigrams after the `<UNK>` mapping, so smoothing can only take mass away from observed words.

1.8.2 Supplementary experiments

The following two experiments use exactly the code of the notebook (same seed and preprocessing); they quantify the claims above. (They are not part of the required output, but they support the discussion of smoothing.)

Training versus test perplexity. Table 6 shows that for $n \geq 3$ the perplexity on the *training* text is much lower than on Hamlet, and the gap grows with n: higher-order models memorize their training data (consistent with the copied phrases in Table 3). For $n = 1$ the training number is higher only because rare training words (counts 2–3) are expensive for a unigram model, whereas many Hamlet tokens fall in the cheap `<UNK>` class; $n = 2$ is close to break-even.

Table 6: Perplexity (Laplace, $k = 1$) on training text versus on Hamlet.

n	Hamlet (test)	Caesar + Macbeth (train)
1	186.5	338.4
2	483.4	512.7
3	1 446.8	922.6
4	1 815.7	997.6
5	1 867.2	1 006.5
10	1 873.2	1 008.2

Smaller smoothing constants. Laplace uses $k = 1$, a very large pseudo-count when $V = 2\,357$. Table 7 shows that add-k smoothing with a smaller k helps the higher-order models substantially (bigram: 483 $\rightarrow$ 240 with $k = 0.05$; trigram: 1 447 $\rightarrow$ 826 with $k = 0.01$), but on a corpus this small they still do not beat the unigram model. (Here k is only varied to illustrate the trend; in a real project it would be chosen on a separate validation set, not on the test set.)

Table 7: Test perplexity on Hamlet for add-k smoothing.

n	$k = 1$	$k = 0.5$	$k = 0.1$	$k = 0.05$	$k = 0.01$
1	186	185	183	183	183
2	483	381	259	240	246
3	1 447	1 284	983	900	826
4	1 816	1 730	1 562	1 507	1 441
5	1 867	1 801	1 680	1 644	1 603

1.9 Part 1 conclusions

Part 1 summary

- We implemented unigram to n-gram models, Laplace smoothing, text generation and perplexity from scratch.
- On 38 000 training tokens the best Hamlet perplexity is the unigram's (186.5); higher-order models suffer from sparsity and from add-one smoothing's over-generous treatment of unseen events (483 for bigrams, about 1 873 for $n \geq 5$).
- Smoothing is indispensable (perplexity would be infinite for $n \geq 2$) but add-one is a crude smoother. Better remedies are more training data, a smaller add-k constant, and above all backoff, interpolation or Kneser–Ney smoothing, which combine lower- and higher-order estimates.
- Perplexities are only comparable between models that use the same preprocessing and vocabulary (lowercase, no punctuation, `min_count`= 2, `<UNK>`, stop-words kept).

2 Part 2: Text Classification

2.1 Dataset

We use the Emotion Recognition dataset `avsolatorio/mteb-emotion-avs_triplets` from Hugging Face. It contains short English sentences labelled with one of six emotions: *sadness, joy, love, anger, fear, surprise*. The dataset is a triplet version of the original emotion data; its columns are `text`, `label`, `label_text`, `idx`, `query_idx`, `positive_idx` and `negative_idx`. We use only `text` and `label`. (The notebook falls back to `dair-ai/emotion`, which has the same sentences and classes, if the triplet dataset lacks these columns; the fallback was not needed.) The 16 000 sentences are split **90/10** into 14 400 training and 1 600 test sentences. The notebook pools all splits that ship with the dataset and re-splits them with a fixed random seed (42), so the split is reproducible. No separate validation set is used: the hyper-parameters (learning rate, number of epochs, smoothing constant) were fixed in advance and the test set was used only for the final evaluation.

Table 8: Dataset splits (left) and class distribution of the test set (right).

Split	Sentences	Test class	Support
Train (90%)	14 400	sadness	475
Test (10%)	1 600	joy	544
		love	125
		anger	211
		fear	174
		surprise	71

The classes are clearly **imbalanced**: *joy* and *sadness* make up 63.7% of the test set (1 019 of 1 600 sentences), while *surprise* has only 71 sentences (4.4%) and *love* 125 (7.8%). This imbalance drives most of the results discussed below.

2.2 Preprocessing for classification

The pipeline mirrors Part 1: lowercase; remove punctuation, digits and special characters with `re.sub(r"[^a-z\s]", " ", text)`; split on whitespace. Stop-word removal is available but **off**, since words such as *not, no, i, so* carry emotional signal. Example:

> *"i do i feel like i just make him mad or upset and he doesn't talk to me"*
> → *[i, do, i, feel, like, i, just, make, him, mad, or, upset, and, he, doesn, t, talk, to, me]*

One difference from Part 1 is that apostrophes are replaced by spaces here, so *doesn't* becomes the two tokens *doesn* and *t*. This is harmless for bag-of-words models because the pieces are still consistent between training and test text, but it does lose the contraction as a single token (a possible refinement).

2.3 Part 2.1: Naïve Bayes classification

2.3.1 Model

Naïve Bayes applies Bayes' rule and assumes that words are conditionally independent given the class. For a sentence d with words $w_1, \ldots, w_m$,

$$\hat{c} = \arg\max_c \Big[\log P(c) + \sum_{i=1}^{m} \log P(w_i \mid c) \Big]. \tag{7}$$

Prior: $P(c) = N_c/N$, where N_c is the number of training sentences of class c and N the total. **Likelihood with Laplace smoothing:**

$$P(w \mid c) = \frac{\text{count}(w, c) + \alpha}{\sum_{w'} \text{count}(w', c) + \alpha |V|}, \qquad \alpha = 1, \tag{8}$$

where $|V| = 14\,453$ is the number of distinct training words. We work in log space (sums of logs instead of products of small numbers, to avoid underflow). Words that were not seen in training are *ignored* at prediction time, which is equivalent to giving them the same likelihood in every class.

2.3.2 Implementation

The class `NaiveBayesNP` stores a $C \times |V|$ count matrix built with `np.add.at`, from which it derives the log-prior vector and the $C \times |V|$ log-likelihood matrix. Prediction sums the relevant columns of the log-likelihood matrix and takes the `argmax`. Fitting and predicting the test set takes 0.12 s.

2.3.3 Comparison with scikit-learn

The reference uses `CountVectorizer` for the bag-of-words representation and `MultinomialNB` for classification, both with $\alpha = 1$. The default `token_pattern` of `CountVectorizer` drops one-letter words, so we changed it to `(?u)\b\w+\b` to keep words such as *i* and to match our preprocessing. Both models are evaluated with the metrics of Part 2.3.

Result

Our from-scratch Naïve Bayes and scikit-learn's `MultinomialNB` produce **identical predictions (100.0% agreement)** and therefore identical confusion matrices and metrics: accuracy 0.7450, macro-precision 0.8411, macro-recall 0.5277, macro-F1 0.5524. This is expected, because both implement the same estimator (multinomial likelihoods, add-one smoothing, the same vocabulary); this also validates our implementation.

Per-class results are in Tables 10–12 (Section 2.6). Naïve Bayes is excellent on the large classes (recall 0.943 for *sadness*, 0.941 for *joy*) but fails on the small ones: *surprise* has recall 0.014 (1 of 71 sentences found) and *love* 0.200. Its precision on these classes is high (*love* 0.833; *surprise* 1.000, but from a single prediction), meaning that when it predicts them it is usually right, but it predicts them very rarely. A plausible reason is that the prior $P(c)$ and the sparser word statistics of the small classes make the large classes win most close decisions.

2.4 Part 2.2: Logistic regression

2.4.1 Feature representation: binary bi-gram presence

Each preprocessed sentence is converted to bi-grams; boundary markers `<s>` and `</s>` are added so that even one-word sentences produce a bi-gram. The vocabulary of all distinct bi-grams is built from the **training** sentences only (building it from the test data would leak information), giving 103 489 features; unseen bi-grams at test time are dropped. A sentence is a sparse binary vector with a 1 at each bi-gram it contains; it has on average 20.0 active features out of 103 489 (a density of about 0.02%), so we store only the indices of the active features.

Discussion: is this a good way of representing features? Advantages

- It captures some local word order and context that unigrams miss, for example negation and intensifiers (*not happy*, *so scared*, *i love*).
- Binary presence is simple, and robust to repeated words (the sentences are short).
- The vectors are extremely sparse, so storage and training are cheap: we only touch about 20 features per sentence.
- Linear models on n-gram features are strong and interpretable baselines: each weight shows how much a phrase votes for a class.

Limitations

- The feature space is huge compared with the data: 103 489 features for 14 400 training sentences, and most bi-grams occur once. The model can overfit, and most weights receive very few updates.
- Data sparsity: a bi-gram not seen in training produces no feature at all, and similar phrasings (*so happy* versus *really glad*) share nothing. There is no notion of synonymy or semantic similarity.
- Only one neighbouring word is used as context; long-range dependencies and the scope of negation are lost.
- Binary presence discards counts and word importance (no TF-IDF weighting).
- The preprocessing removes some emotional cues (punctuation such as "!", digits, apostrophes).

Better alternatives include combining unigrams and bi-grams, a minimum-frequency cutoff, TF-IDF weighting, or dense embeddings.

2.4.2 Softmax (multinomial) logistic regression from scratch

For a batch of B sentences, each a sparse binary vector x_i, the four steps required by the lab are:

Step 1: scores $z_i = x_i W + b = \sum_{j \in \text{active}(i)} W[j] + b$. Only active features are summed, which equals the full product $XW + b$ but is much faster.

Step 2: probabilities and loss

$$P_{i,k} = \frac{e^{z_{i,k}}}{\sum_m e^{z_{i,m}}}, \qquad L = -\frac{1}{B} \sum_{i=1}^{B} \log P_{i,y_i}. \tag{9}$$

The row maximum is subtracted before exponentiation for numerical stability (the result is unchanged).

Step 3: gradients With Y the one-hot labels,

$$dZ = \frac{P - Y}{B}, \qquad dW[j] = \sum_{i:\, j \in \text{active}(i)} dZ_i, \qquad db = \sum_{i=1}^{B} dZ_i. \tag{10}$$

Step 4: update $W \leftarrow W - \eta\, dW$, $b \leftarrow b - \eta\, db$.

Training setup. Weights and bias are initialized to zero (acceptable for softmax regression because the loss is convex); mini-batch SGD with batch size $B = 64$, constant learning rate $\eta = 0.5$ and 15 epochs; the training set is **shuffled at the start of every epoch**. Training takes 1.0 s. Since there is no validation set, we monitor only the training loss, shown per epoch in Figure 2.

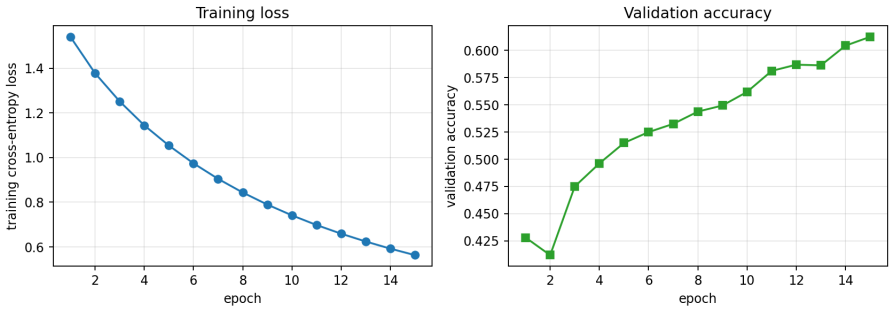


Figure 2: Training of the from-scratch logistic regression. The training loss falls from 1.536 (epoch 1) to 0.551 (epoch 15) and has not flattened by the last epoch.

2.4.3 Comparison with scikit-learn

We compare against `sklearn.linear_model.LogisticRegression` (default L2 regularisation $C = 1$, L-BFGS solver, `max_iter=1000`) and `SGDClassifier(loss="log_loss", max_iter=50, tol=1e-4)`, using exactly the same binary bi-gram features, so only the model/optimiser/regulariser differ.

Table 9: How the three logistic-regression implementations differ.

	Ours (NumPy)	`LogisticRegression`	`SGDClassifier`
Objective	single multinomial softmax, cross-entropy	multinomial softmax, cross-entropy + L2	log-loss, one-vs-rest binary classifiers + L2
Optimiser	mini-batch SGD ($B = 64$)	L-BFGS, deterministic, run to convergence	SGD on one sample at a time
Learning rate	constant, 0.5	none (line search)	automatic schedule
Regularisation	none	L2 ($C = 1$)	small L2 (default)
Shuffling	every epoch	not applicable	every epoch
Stopping	fixed 15 epochs	tolerance / 1000 iterations	tolerance / 50 epochs
Training time	1.0 s	5.1 s	0.1 s

Which approach performs better, and under what conditions? By accuracy the order is Naïve Bayes (0.7450) > `LogisticRegression` (0.6825) ≈ `SGDClassifier` (0.6806) > our logistic regression (0.6319). By macro-F1, which weighs every emotion equally, the order is different: `SGDClassifier` (0.5686) > `LogisticRegression` (0.5597) > Naïve Bayes (0.5524) > ours (0.4614). The discriminative models trained on bi-grams find more of the rare classes, especially *surprise* (recall 0.183 for `LogisticRegression` and 0.225 for `SGDClassifier`, versus 0.014 for Naïve Bayes); for *love* the gain is small (0.208 and 0.224 versus 0.200). This is why they win on macro-F1 although they lose on accuracy. The conditions matter:

- **Naïve Bayes** is the best choice when the goal is overall accuracy, when training time must be minimal (one counting pass), or when data is scarce; its weakness is the rare classes.
- **Regularised, converged logistic regression** handles the very high-dimensional sparse bi-gram features better: the L2 term limits overfitting, and the solver finds the optimum of the convex loss.
- **Our SGD version** is clearly behind scikit-learn (5.1 accuracy points and 9.8 macro-F1 points below `LogisticRegression`). Its training loss is still decreasing at epoch 15 (from 0.579 to 0.551 in the last epoch, Figure 2), and it has no regularisation and a constant learning rate, so it is probably *under-trained*. We expect more epochs, a decaying learning

rate or an L2 term to narrow the gap, but we did not test this; because no validation set is used, such tuning would require splitting one off from the training data rather than using the test set.
- The two scikit-learn models are very close (0.9 points of macro-F1 apart). With only 71 *surprise* sentences in the test set, a single sentence changes the *surprise* recall by 1.4 points, so this small difference should not be over-interpreted.

Which implementation is most similar to ours? In terms of *optimisation procedure*, `SGDClassifier` is the closest: both minimise a logistic (cross-entropy) loss with stochastic gradient updates and shuffle the data each epoch. The differences are that `SGDClassifier` updates on single samples rather than mini-batches, trains one-vs-rest binary classifiers instead of one multinomial softmax, adds a small L2 penalty, and uses an automatic learning-rate schedule. In terms of the *model and objective*, `LogisticRegression` is the closest (the same multinomial softmax regression), but it uses a deterministic second-order-type solver (L-BFGS) with L2 regularisation. Empirically the two are equally close to our predictions: our model agrees with `LogisticRegression` on 88.6% of the test sentences and with `SGDClassifier` on 88.8%. We therefore consider `SGDClassifier` the most similar *implementation* and `LogisticRegression` the most similar *model*.

2.5 Part 2.3: Confusion matrix and evaluation metrics

2.5.1 Definitions

For K classes, the confusion matrix $CM \in \mathbb{N}^{K\times K}$ has rows indexed by the true class and columns by the predicted class: CM_{ij} counts the samples of true class i predicted as class j. For class k:

$$TP_k = CM_{kk}, \qquad FP_k = \Big(\sum_i CM_{ik}\Big) - TP_k, \qquad FN_k = \Big(\sum_j CM_{kj}\Big) - TP_k,$$

$$\text{i.e. } FP_k = \text{column sum} - TP_k, \qquad FN_k = \text{row sum} - TP_k,$$

$$P_k = \frac{TP_k}{TP_k + FP_k}, \qquad R_k = \frac{TP_k}{TP_k + FN_k}, \qquad F1_k = \frac{2P_kR_k}{P_k + R_k}. \tag{11}$$

The **macro average** is the unweighted mean over classes,

$$P_{\text{macro}} = \frac{1}{K}\sum_k P_k, \quad R_{\text{macro}} = \frac{1}{K}\sum_k R_k, \quad F1_{\text{macro}} = \frac{1}{K}\sum_k F1_k, \tag{12}$$

so every class counts equally regardless of its size. Note that macro-F1 is the mean of the per-class F1 values, not the F1 of the macro-precision and macro-recall. (For comparison, the *micro* average pools all TP/FP/FN, and in single-label multi-class classification it equals accuracy; the *weighted* average weighs each class by its support, so the large classes dominate.)

2.5.2 Implementation

Using only NumPy: `confusion_matrix_np` fills the matrix with `np.add.at(cm, (y_true, y_pred), 1)`; `prf_from_cm` obtains TP, FP and FN from the diagonal, column sums and row sums, and computes precision, recall and F1 with a safe division that returns 0 when the denominator is 0 (the same convention as scikit-learn's `zero_division=0`, needed when a class is never predicted); `evaluate` also returns the accuracy $\text{tr}(CM)/\sum CM$ and the macro averages.

Worked example with real numbers. From the Naïve Bayes confusion matrix (Figure 3): for *love*, $TP = 25$, the column sum is $1 + 4 + 25 + 0 + 0 + 0 = 30$ so $FP = 5$, and the row sum is 125 so $FN = 100$. Hence $P = 25/30 = 0.833$, $R = 25/125 = 0.200$ and $F1 = 2(0.833)(0.200)/(0.833 + 0.200) = 0.323$, exactly the values in the tables. For *surprise*: $TP = 1$, column sum $0 + 0 + 0 + 0 + 0 + 1 = 1$ so $FP = 0$, row sum 71 so $FN = 70$; $P = 1/1 = 1.000$, $R = 1/71 = 0.014$, $F1 = 0.028$. The macro-F1 is $(0.818+0.813+0.323+0.674+0.659+0.028)/6 = 0.552$.

2.5.3 Validation against scikit-learn

We compare our results with `sklearn.metrics.confusion_matrix`, `precision_score`, `recall_score` and `f1_score` (per class with `average=None` and macro with `average="macro"`, `zero_division=0`). The notebook checks the equality of the confusion matrix (exactly) and of all per-class and macro precision, recall and F1 values (with `np.allclose`) on random predictions, so that the check is independent of any model; it passes. Since the confusion matrix is compared exactly and every other metric is derived from it with the same formulas, agreement on the matrix and on the metrics implies that our metric implementation is correct. As a further consistency check on real predictions, our evaluator gives identical tables for our Naïve Bayes and for scikit-learn's `MultinomialNB`, whose predictions agree on 100% of the test sentences.

2.6 Results on the test set

Tables 10, 11 and 12 give the per-class precision, recall and F1 of each model, Table 13 the summary, and Figures 3 and 4 the confusion matrices and per-class F1. The scikit-learn Naïve Bayes results are identical to ours and are therefore not repeated.

Table 10: Per-class precision on the test set.

Class	Support	NB	LR (ours)	LR (sklearn)	SGD (sklearn)
sadness	475	0.721	0.640	0.670	0.717
joy	544	0.715	0.587	0.653	0.614
love	125	0.833	0.524	0.605	0.622
anger	211	0.885	0.848	0.812	0.847
fear	174	0.892	0.787	0.797	0.816
surprise	71	1.000	0.600	0.722	0.727
Macro	1600	**0.8411**	**0.6642**	**0.7096**	**0.7239**

Table 11: Per-class recall on the test set.

Class	Support	NB	LR (ours)	LR (sklearn)	SGD (sklearn)
sadness	475	0.943	0.777	0.802	0.758
joy	544	0.941	0.869	0.857	0.895
love	125	0.200	0.088	0.208	0.224
anger	211	0.545	0.370	0.531	0.498
fear	174	0.523	0.425	0.540	0.534
surprise	71	0.014	0.085	0.183	0.225
Macro	1600	**0.5277**	**0.4356**	**0.5201**	**0.5224**

Table 12: Per-class F1-score on the test set.

Class	Support	NB	LR (ours)	LR (sklearn)	SGD (sklearn)
sadness	475	0.818	0.702	0.730	0.737
joy	544	0.813	0.701	0.741	0.728
love	125	0.323	0.151	0.310	0.329
anger	211	0.674	0.515	0.642	0.627
fear	174	0.659	0.552	0.644	0.646
surprise	71	0.028	0.148	0.292	0.344
Macro	1600	**0.5524**	**0.4614**	**0.5597**	**0.5686**

Table 13: Final comparison on the test set (1 600 sentences). Best value of each column in bold.

Model	Accuracy	Macro-P	Macro-R	Macro-F1	Fit time
Naïve Bayes (ours)	**0.7450**	**0.8411**	0.5277	0.5524	0.12 s
Naïve Bayes (sklearn)	**0.7450**	**0.8411**	0.5277	0.5524	–
Logistic regression (ours)	0.6319	0.6642	0.4356	0.4614	1.0 s
Logistic regression (sklearn)	0.6825	0.7096	0.5201	0.5597	5.1 s
SGDClassifier (sklearn)	0.6806	0.7239	**0.5224**	**0.5686**	0.1 s

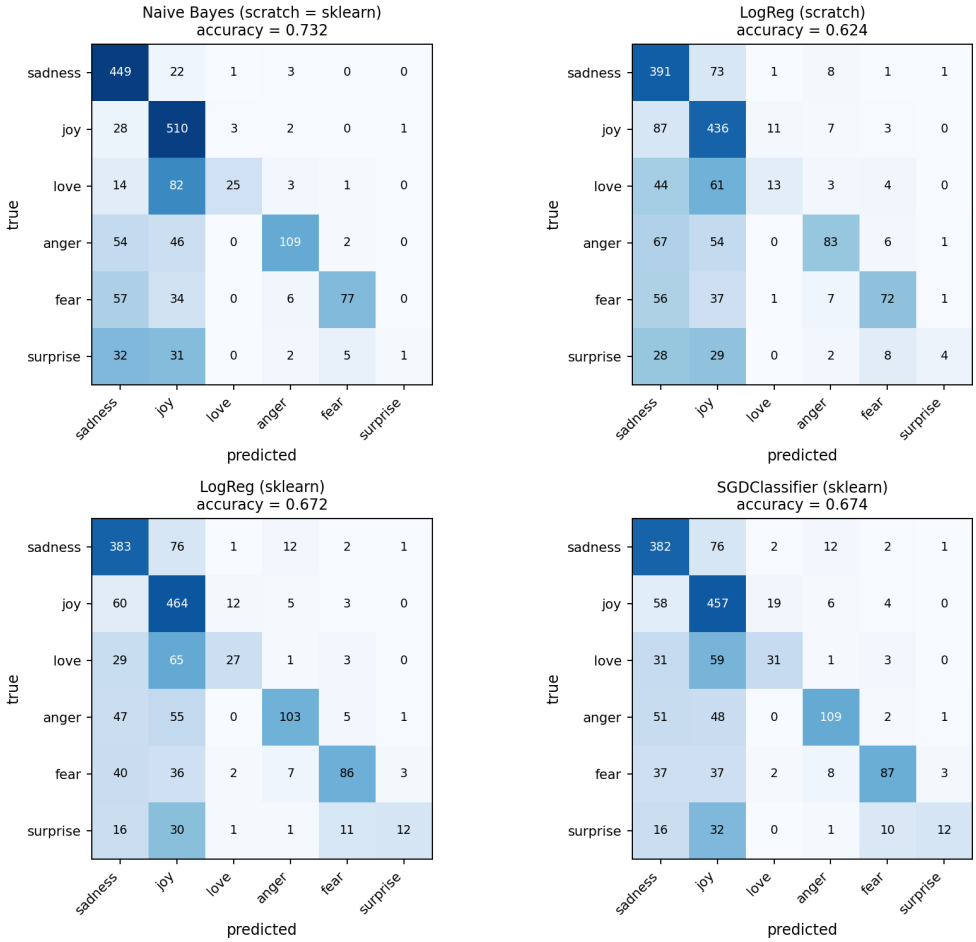


Figure 3: Confusion matrices on the test set (rows: true class, columns: predicted class). Cell shading is the row-normalised value (the diagonal is the recall) and the number is the raw count. Row normalisation matters because of the class imbalance.

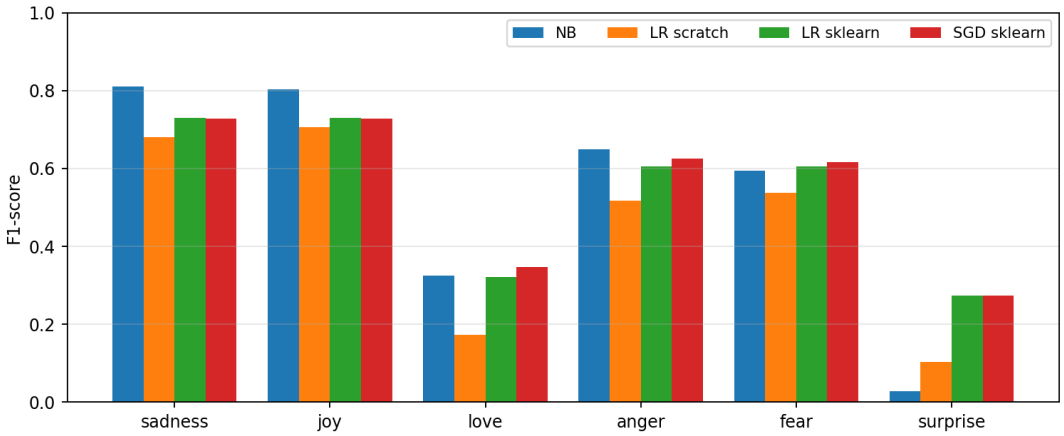


Figure 4: Per-class F1-score of the four distinct models. All models do well on *sadness* and *joy* and poorly on *love* and *surprise*.

2.7 Discussion of the classification results

Accuracy versus macro-averaging. The rankings by accuracy and by macro-F1 disagree. Naïve Bayes has the highest accuracy because *joy* and *sadness* contain 63.7% of the test sentences and Naïve Bayes is very good on them (recall about 0.94). But it recovers only 1 of 71 *surprise* sentences and 25 of 125 *love* sentences, so its macro-recall (0.528) is far below its accuracy (0.745). Since macro-averaging gives each of the six emotions the same weight, it exposes this weakness; accuracy hides it. For an imbalanced problem where all emotions matter, macro-F1 is therefore the fairer headline metric. Its price is that it is sensitive to the smallest classes (a few sentences of *surprise* move it noticeably), so it is best reported together with the per-class tables and the confusion matrices.

Where the errors go. In every confusion matrix the large mass of errors goes into *sadness* and *joy*: *love* is mostly mistaken for *joy* (80 of 125 sentences for Naïve Bayes, 62 for `LogisticRegression`), and *anger*, *fear* and *surprise* are mostly mistaken for *sadness* or *joy*. These are semantically neighbouring emotions with overlapping vocabulary, which a bag-of-words or bi-gram representation cannot separate well; a classifier that is uncertain also tends to fall back on the most frequent classes.

Generative versus discriminative. Naïve Bayes (generative, independence assumption) is extremely cheap and strong on frequent classes, but its decisions are dominated by priors and frequent words. Logistic regression (discriminative) can give rare-but-informative features, such as particular bi-grams, large weights, and recovers more of the rare classes, but it needs optimisation (and regularisation) to do so.

Part 2 summary

- Data split: 90% train (14 400) / 10% test (1 600), with no separate validation set.
- Naïve Bayes: our implementation equals scikit-learn's exactly; best accuracy (0.7450) but weak on *love* and *surprise* (macro-F1 0.5524).
- Logistic regression on 103 489 binary bi-gram features: our NumPy version reaches 0.6319 accuracy and 0.4614 macro-F1 after 15 epochs, with the training loss still falling; scikit-learn's versions reach 0.6825/0.5597 (`LogisticRegression`) and 0.6806/0.5686 (`SGDClassifier`), the best macro-F1 overall.
- Our confusion-matrix, precision, recall and F1 code (per class and macro) matches scikit-learn exactly.
- Because the classes are imbalanced, macro-averaged metrics and per-class tables are essential for an honest comparison.